Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Военный факультет
Кафедра: Электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО. КОНВЕРТЕР ВАЛЮТ»
Вариант 1

Студент: Г.А. Гамза

Преподаватель: С.С. Силич

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки Visual Studio Code при создании объектно-ориентированного приложения на языке C++, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 1 необходимо разработать утилиту для конвертации белорусского рубля (BYN) в евро (EUR). Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект C++ в VS Code, изучить средства редактирования, сборки, запуска и отладки;
2. Подобрать средства VS Code для работы с UML и отразить результат настройки в отчёте;
3. Разработать графическое приложение с главной формой и отдельным окном ввода сигналов;
4. Обеспечить сохранение последних корректных данных и их восстановление при повторном вводе;
5. Реализовать проверку входных данных и вывод понятного сообщения при ошибке;
6. Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью;
7. Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки объединяет редактор исходного кода, средства навигации и рефакторинга, систему сборки, запуск приложения, отладчик, терминал и интеграцию с системой контроля версий. . Visual Studio Code (VS Code) – легковесный кроссплатформенный редактор с богатой экосистемой расширений. Для разработки C++-приложений устанавливаются расширения C/C++ (Microsoft), C/C++ Extension Pack, а также компилятор (MSVC, MinGW-w64 или Clang) и отладчик.

2.2 Редактирование и навигация в Visual Studio Code

Редактор VS Code поддерживает автодополнение (IntelliSense), форматирование, безопасное переименование (F2), поиск использований (Shift+F12), переход к определению (F12). Для форматирования используется сочетание Shift+Alt+F, для вызова команд – Ctrl+Shift+P. Структура проекта отображается в боковой панели Explorer, сообщения компилятора и программы – во вкладках Terminal и Debug Console.

2.3 Сборка, запуск и отладка C++-приложения

Исходные файлы C++ компилируются компилятором (cl.exe, g++ или clang++) в исполняемый файл. В VS Code проект собирается через задачи (tasks.json) или внешнюю систему сборки (CMake, MSBuild). Конфигурация запуска и отладки задаётся в файле .vscode/launch.json. Точка останова устанавливается щелчком слева от номера строки или клавишей F9. Во время остановки окно Variables показывает значения переменных, Call Stack – стек вызовов. Команды Step Over (F10), Step Into (F11) и Step Out (Shift+F11) позволяют выполнять программу пошагово.

2.4 Паттерн MVC

MVC (Model–View–Controller) разделяет приложение на три части. Model хранит состояние и выполняет предметную логику. View формирует интерфейс и отображает данные. Controller обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов и позволяет независимо развивать и тестировать компоненты.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сообщает подписчикам об изменении состояния. В разработанном приложении CurrencyModel использует паттерн «Наблюдатель» (Observer) через интерфейс IModelObserver: после успешного вызова SetData модель сама вызывает NotifyObservers(), и представление обновляет метки в методе OnModelChanged().

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана утилита, которая принимает сумму в белорусских рублях (BYN) и курс евро (BYN за 1 EUR) и вычисляет эквивалент в евро. На главной форме размещена кнопка «Ввод чисел», открывающая модальное окно InputDialog. Приложение использует чистый WinAPI (без MFC/Qt), стилизованный интерфейс с карточкой, кастомными кнопками и шрифтом Segoe UI. Реализована поддержка восстановления данных: при открытии окна ввода подставляются последние успешно обработанные значения.

3.2 Настройка среды и расширений для UML и SysML v2

В Visual Studio Code создан проект с исходными файлами (.cpp, .h), файлом манифеста app.manifest и resource.rc. Установлены расширения C/C++ (Microsoft) и C/C++ Extension Pack. Для сборки используется MSVC (или MinGW). Конфигурации tasks.json и launch.json позволяют собирать и запускать приложение, а также выполнять отладку. Для UML применяется расширение PlantUML или Draw.io Integration; диаграмма классов описана в текстовом виде.

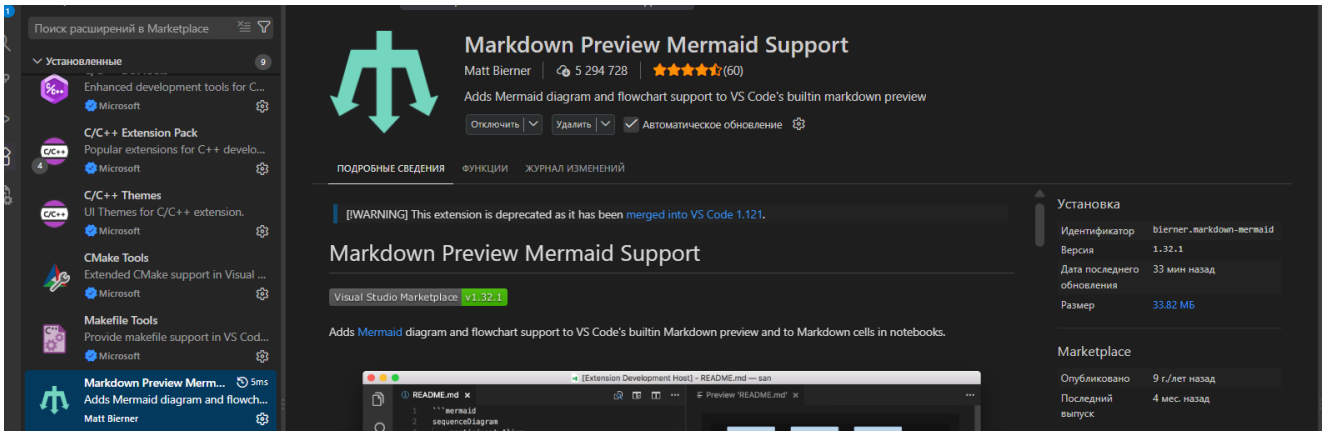


Рисунок 3.1 – Средства VS Code для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2/ UML:

```
package CurrencyConverterMVC {
  class CurrencyModel {
    +SetData(sumByn, rateEur)
    +GetResult() : double
    +AddObserver(IModelObserver*)
    -NotifyObservers()
  }
  class MainWindow <<View>> {
    +OnModelChanged()
```

```
  +Create()
}
class AppController <<Controller>> {
  +OnInputButtonClicked(HWND)
}
class InputDialog {
  +Show(...) : bool
}
interface IModelObserver {
  +OnModelChanged()
}
interface IController {
  +OnInputButtonClicked(HWND)
}
MainWindow ..|> IModelObserver
AppController ..|> IController
CurrencyModel --> IModelObserver : notifies
MainWindow --> IController
AppController --> CurrencyModel
AppController --> InputDialog
```

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе SetData класса CurrencyModel. Точка останова устанавливается на строке вычисления result = sumByn / rateEur. В окне Variables можно проследить значения lastSum, lastRate и result, а также проверить срабатывание NotifyObservers(). Функция Evaluate Expression позволяет вычислить произвольные выражения во время остановки.

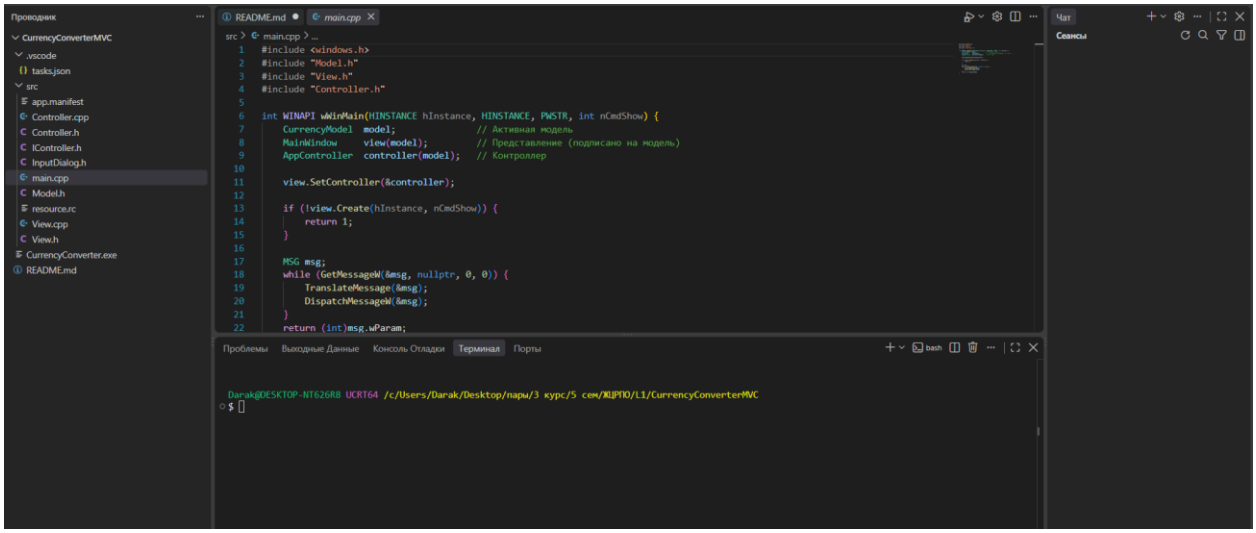


Рисунок 3.2 – Отладчик VS Code

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- CurrencyModel (Model.h) – хранит сумму, курс и результат конвертации, содержит бизнес-логику и механизм уведомления наблюдателей;
- MainWindow (View.h / View.cpp) – главное окно, реализует IModelObserver для автоматического обновления при изменении данных;
- InputDialog (InputDialog.h) – модальное окно для ввода суммы и курса (чистое динамическое создание окон WinAPI);
- AppController (Controller.h / Controller.cpp) – обрабатывает нажатие кнопки, открывает диалог и передаёт данные в модель;
- IController / IModelObserver – интерфейсы для слабой связанности;
- main.cpp – точка входа, создаёт модель, представление и контроллер и запускает цикл сообщений.

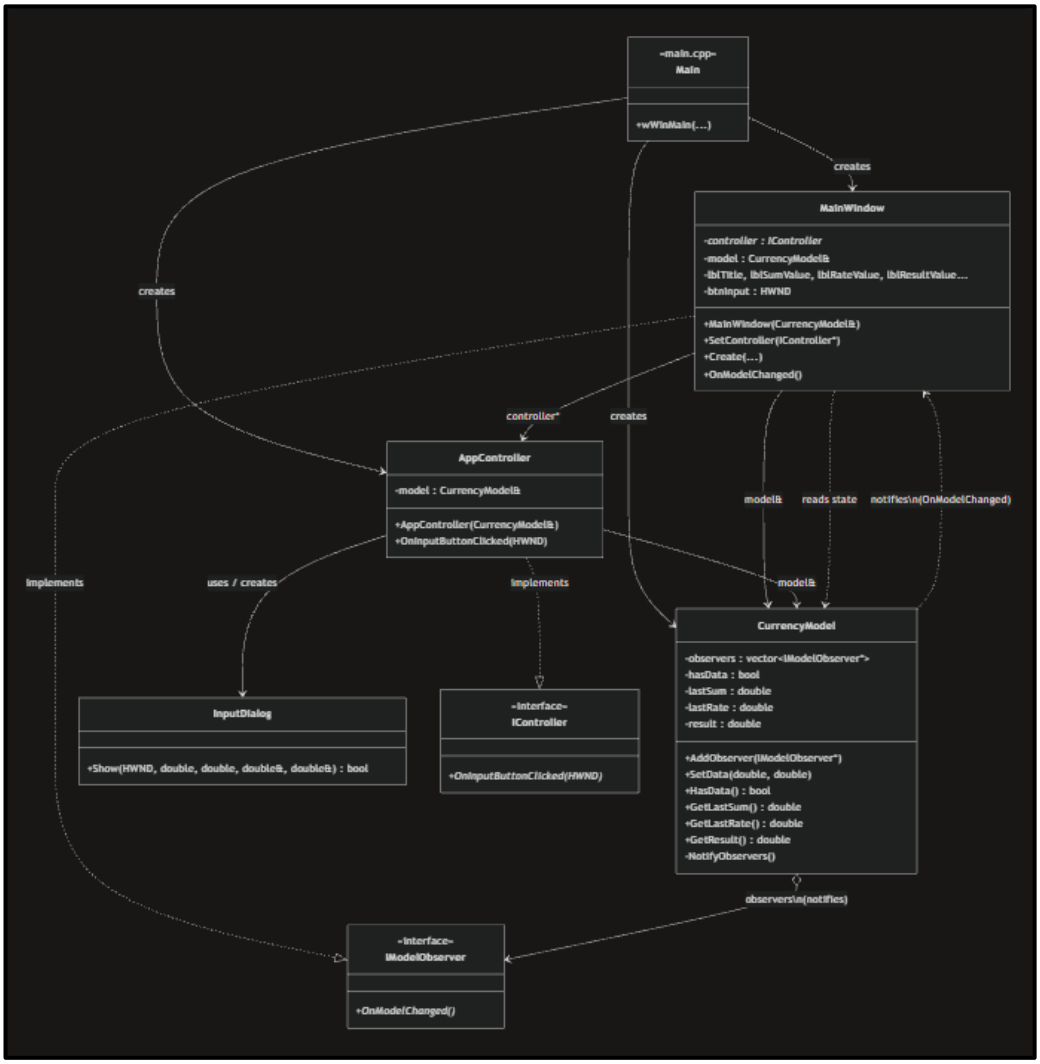


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

CurrencyModel реализует активную модель. Метод SetData выполняет проверку: сумма и курс должны быть строго положительными числами. При некорректных данных бросается std::invalid_argument. После успешной валидации вычисляется результат (BYN / курс = EUR), сохраняются последние значения и вызывается NotifyObservers(). Представление, подписанное через AddObserver, получает уведомление OnModelChanged() и обновляет метки.

3.6 Представление

MainWindow отображает текущую сумму, курс и результат конвертации в стилизованной карточке. Благодаря реализации интерфейса IModelObserver метод OnModelChanged вызывает UpdateLabels(), который форматирует числа и обновляет STATIC-контролы. Кнопка «Ввод чисел» (owner-draw) передаёт управление контроллеру. InputDialog обеспечивает модальный ввод с поддержкой восстановления предыдущих значений и проверкой корректности чисел (включая замену запятой на точку).

3.7 Контроллер

AppController управляет жизненным циклом InputDialog. При нажатии кнопки он передаёт в диалог последние значения из модели. После успешного ввода вызывает model.SetData(). Исключения, возникающие в модели, перехватываются и отображаются пользователю через MessageBox. Контроллер не обновляет View напрямую – это делает сама активная модель.

3.8 Запоминание данных и обработка ошибок

Приложение предотвращает ввод некорректных (неположительных или нечисловых). данных. Сообщения об ошибках формируются как в диалоге ввода («Введите корректные положительные числа в оба поля»), так и в модели («Сумма и курс должны быть положительными числами»). Восстановление данных реализовано передачей model.GetLastSum() и model.GetLastRate() в InputDialog::Show при каждом открытии

4 РЕЗУЛЬТАТЫ РАБОТЫ

В VS Code создано и проверено C++-приложение варианта 1 Программа выполняет конвертацию, сохраняет последнее успешное состояние и сообщает о некорректном вводе. Архитектура соответствует MVC с активной моделью: контроллер обрабатывает пользовательское действие, модель выполняет расчёт и уведомляет представление через интерфейс IModelObserver.

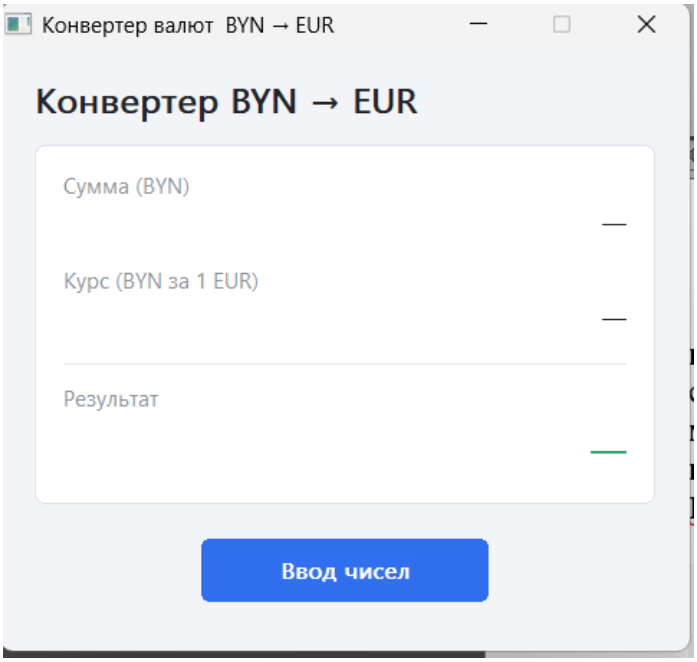


Рисунок 4.1 – Главная форма после запуска

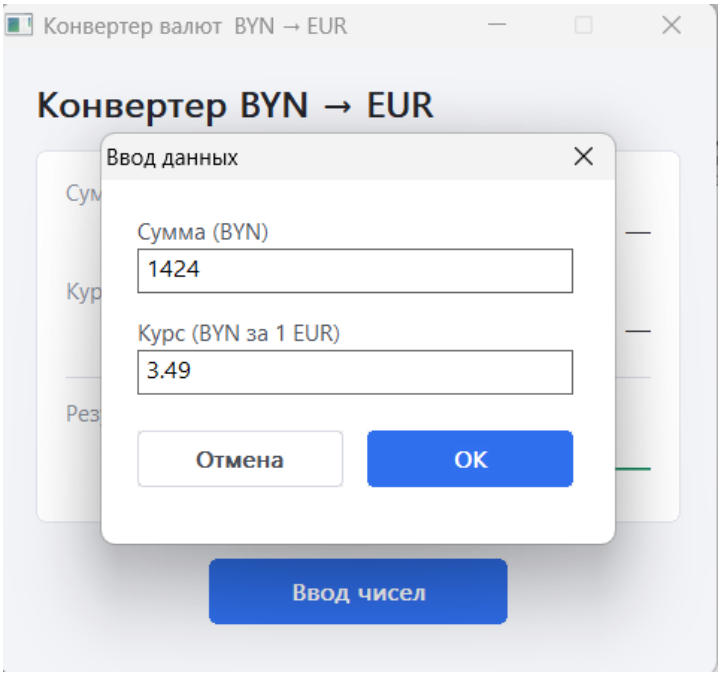


Рисунок 4.2 – Окно ввода данных

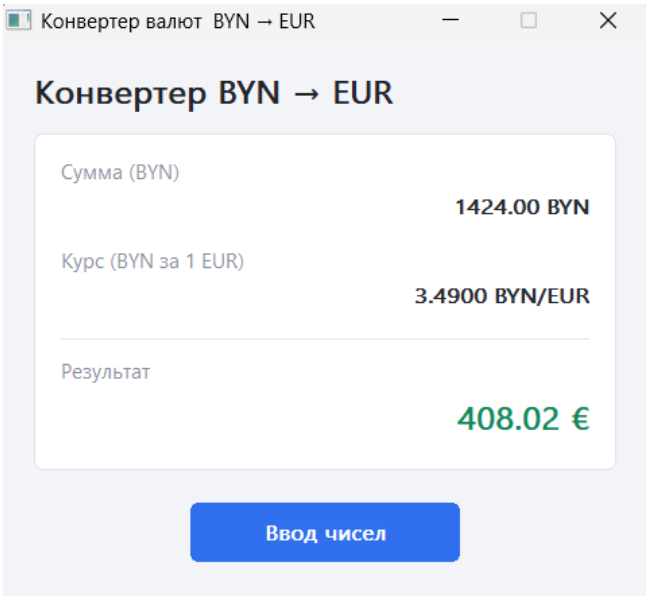


Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор/линкер, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений. В Visual Studio Code основные рабочие области – Explorer, Search, Source Control, Run and Debug, Extensions, а также панели Terminal, Problems и Output.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение (IntelliSense), быстрые исправления (Quick Fix), форматирование, переход к определению (F12), поиск использований (Shift+F12), переименование (F2), просмотр иерархии вызовов и анализ предупреждений. Эти операции позволяют изменять код быстрее и уменьшают вероятность механических ошибок.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

В VS Code для C++ настраиваются tasks.json (сборка) и launch.json (запуск/отладка). Можно использовать CMake Tools или вызывать компилятор напрямую (cl / EHsc / std:c++17 *.cpp /link user32.lib gdi32.lib comctl32.lib). После успешной сборки исполняемый файл запускается командой Run или Debug. При запуске в режиме Debug программа останавливается на заданных точках останова.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет MorseDecoderModel, роль представления – MainView и InputDialog, а роль контроллера – MorseController.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому после её изменения контроллер должен отдельно обновить интерфейс. Активная модель публикует событие при изменении своего состояния. В приложении CurrencyModel вызывает NotifyObservers(), а MainWindow получает уведомление через метод OnModelChanged() и самостоятельно обновляет элементы формы.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности Visual Studio Code для редактирования, сборки, запуска и отладки C++-приложения с графическим интерфейсом WinAPI. Рассмотрены средства работы с UML и подготовлена диаграмма классов приложения.

Для варианта разработана утилита конвертации BYN → EUR. Программа поддерживает ввод положительных вещественных чисел, сохраняет последние корректные значения и выводит понятные сообщения об ошибках. Компоненты разделены по паттерну MVC, а активная модель уведомляет представление о новом состоянии с помощью интерфейса IModelObserver (паттерн Observer).

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

main.cpp

```
#include <windows.h>
#include "Model.h"
#include "View.h"
#include "Controller.h"

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int nCmdShow)
{
    CurrencyModel  model;
    MainWindow     view(model);
    AppController  controller(model);

    view.SetController(&controller);

    if (!view.Create(hInstance, nCmdShow)) {
        return 1;
    }

    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }
    return (int)msg.wParam;
}
```

Model.h

```
#pragma once
#include <vector>
#include <stdexcept>

// ============================================================
//  Интерфейс наблюдателя. View реализует этот интерфейс, чтобы
//  Model могла оповещать его об изменении данных.
// ============================================================
class IModelObserver {
public:
    virtual void OnModelChanged() = 0;
    virtual ~IModelObserver() {}
};

// ============================================================
//  АКТИВНАЯ МОДЕЛЬ (Active Model)
//  Модель сама содержит бизнес-логику (конвертацию валют) и
```

```
//  САМА оповещает подписчиков (View) о своём изменении.
//  Controller лишь передаёт данные от пользователя в модель,
//  но не занимается обновлением View напрямую - это делает
//  сама модель через рассылку уведомлений (Observer pattern).
// ============================================================
class CurrencyModel {
public:
    void AddObserver(IModelObserver* observer) {
        observers.push_back(observer);
    }

    // Установить новые исходные данные и выполнить конвертацию.
    // При некорректных данных бросает std::invalid_argument.
    void SetData(double sumByn, double rateEur) {
        if (sumByn <= 0.0 || rateEur <= 0.0) {
            throw std::invalid_argument(
                "Сумма и курс должны быть положительными числами");
        }

        lastSum  = sumByn;
        lastRate = rateEur;
        result   = sumByn / rateEur;   // BYN -> EUR
        hasData  = true;

        NotifyObservers();   // <-- активная модель сама уведомляет View
    }

    bool   HasData()     const { return hasData; }
    double GetLastSum()  const { return lastSum; }
    double GetLastRate() const { return lastRate; }
    double GetResult()   const { return result; }

private:
    void NotifyObservers() {
        for (auto* observer : observers) {
            if (observer) observer->OnModelChanged();
        }
    }

    std::vector<IModelObserver*> observers;

    bool   hasData  = false;
    double lastSum  = 0.0;
    double lastRate = 0.0;
    double result   = 0.0;
};
```

Controller.cpp

```
#include "Controller.h"
#include "InputDialog.h"
#include <stdexcept>

void AppController::OnInputButtonClicked(HWND parentWnd) {
    double sum  = model.HasData() ? model.GetLastSum()  : 0.0;
    double rate = model.HasData() ? model.GetLastRate() : 0.0;

    double newSum = 0.0, newRate = 0.0;
    if (InputDialog::Show(parentWnd, sum, rate, newSum, newRate)) {
        try {
            model.SetData(newSum, newRate);   // модель сама уведомит
View
        } catch (const std::exception& ex) {
            MessageBoxA(parentWnd, ex.what(), "Ошибка", MB_OK |
MB_ICONERROR);
        }
    }
}
```

View.h

```
#pragma once
#include <windows.h>
#include "Model.h"

class IController;

// ============================================================
//  VIEW: главное окно приложения (стилизованный UI: карточка,
//  акцентные цвета, шрифт Segoe UI, кастомная кнопка).
//  Реализует IModelObserver -> подписывается на модель и
//  обновляет метки, когда модель сама сообщает об изменении.
// ============================================================
class MainWindow : public IModelObserver {
public:
    explicit MainWindow(CurrencyModel& model) : model(model) {
        model.AddObserver(this);
    }

    void SetController(IController* c) { controller = c; }

    bool Create(HINSTANCE hInstance, int nCmdShow);

    HWND GetHwnd() const { return hwnd; }
```

```
        // IModelObserver
        void OnModelChanged() override { UpdateLabels(); }

    private:
        static LRESULT CALLBACK WndProcStatic(HWND hWnd, UINT msg, WPARAM
wParam, LPARAM lParam);
        LRESULT WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam);

        void CreateFonts();
        void CreateBrushes();
        void CreateControls(HWND hWnd);
        void UpdateLabels();
        void PaintBackground(HWND hWnd, HDC hdc);
        void DrawInputButton(LPDRAWITEMSTRUCT dis);
        void Cleanup();

        CurrencyModel& model;
        IController* controller = nullptr;

        HWND hwnd = nullptr;

        // заголовок
        HWND lblTitle = nullptr;

        // подписи (серые) и значения (тёмные) внутри карточки
        HWND lblCaptionSum = nullptr, lblSumValue = nullptr;
        HWND lblCaptionRate = nullptr, lblRateValue = nullptr;
        HWND lblCaptionResult = nullptr, lblResultValue = nullptr;

        HWND btnInput = nullptr;

        // геометрия карточки (используется и в WM_PAINT, и при
        // расстановке контролов)
        RECT cardRect{};
        int dividerY = 0;

        // шрифты
        HFONT fontTitle   = nullptr;
        HFONT fontCaption = nullptr;
        HFONT fontValue   = nullptr;
        HFONT fontResult  = nullptr;
        HFONT fontButton  = nullptr;

        // кисти/цвета
        HBRUSH hbrBackground = nullptr; // светлый фон окна
        HBRUSH hbrCard       = nullptr; // белый фон карточки
        COLORREF colorBg       = RGB(0xF3, 0xF4, 0xF8);
        COLORREF colorCard     = RGB(0xFF, 0xFF, 0xFF);
```

```
    COLORREF colorBorder   = RGB(0xE2, 0xE4, 0xEC);
    COLORREF colorTitle    = RGB(0x22, 0x24, 0x2A);
    COLORREF colorCaption  = RGB(0x8A, 0x8F, 0x9C);
    COLORREF colorValue    = RGB(0x2A, 0x2C, 0x33);
    COLORREF colorResult   = RGB(0x15, 0x8A, 0x5C);
    COLORREF colorAccent   = RGB(0x2F, 0x6F, 0xED);
    COLORREF colorAccentDn = RGB(0x25, 0x59, 0xC2);
};
```